

Domande e risposte Sicurezza e Affidabilità

Come valuto la giusta difesa?

Quando parliamo di security, a differenza di safety, vi è la presenza di qualcuno chiamato **attaccante** che ragiona su quali sono le vulnerabilità, le cose che il sistema lascia un po' aperte. I requisiti per la sicurezza consistono nella **CIA Triad**:

- **Confidentiality**: la confidenzialità fa riferimento al fatto che una risorsa può essere accessibile solo dietro autorizzazione; si controlla e assicura la **riservatezza** delle informazioni rispetto a soggetti non autorizzati.
Le compromissioni (**Disclosure**) possono riguardare ad esempio l'accesso a risorse il cui contenuto non deve essere noto o l'analisi del traffico di una rete.
- **Integrity**: con l'integrità, invece, la risorsa non deve essere alterabile, quindi nessuno con un attacco può fare delle modifiche; si controlla e assicura l'**attendibilità**.
Le compromissioni (**Alteration**) possono essere la modifica o creazione non autorizzata di un oggetto o la manomissione di informazioni sensibili.
- **Availability**: se una risorsa deve essere disponibile agli utenti di un sistema, devo avere garanzie sul fatto che gli utenti previsti potranno effettivamente accedere; si controlla e assicura la **possibilità di usare risorse** da parte di utenti autorizzati.
Le compromissioni (**Destruction**) possono riguardare DoS, manomissione degli algoritmi di scheduling, compromissione di un file di password ma anche la distruzione fisica di un computer.

Poiché c'è competizione tra usabilità e sicurezza, dobbiamo trovare il giusto compromesso ed è importante che il progetto sia nativamente creato considerando le problematiche di sicurezza. Una corretta analisi dei rischi è fondamentale, in generale con rischio si intende la possibilità di perdita o danno ma in sicurezza informatica $\text{Rischio} = \text{Vulnerabilità} * \text{Minacce} * \text{Valori}$. I primi interventi di security vanno fatti sugli elementi con rischio più alto.

Considerando l'applicazione della crittografia ASIMMETRICA in combinazione con MESSAGE DIGEST per la realizzazione di firme digitali efficienti, e considerando l'utente A che possiede la coppia di chiavi A_priv e A_pub, si descrivano:

- gli step che A deve seguire per inviare un file F firmato all'utente B

- gli step che l'utente B deve seguire per convalidare la veridicità della firma quando riceve il file F

- A calcola il message digest del messaggio, lo cripta con A_priv e lo spedisce insieme al messaggio in chiaro
- B riceve il messaggio in chiaro e la firma cifrata dal mittente
- B la decifra con A_pub e ottiene il message digest del messaggio originale
- B confronta il message digest del messaggio con quello decifrato dalla firma.

Si spieghi il funzionamento della crittografia a chiave pubblica, indicando in particolare come si può usarla per implementare firme digitali

Nella crittografia asimmetrica (a chiave pubblica) sono coinvolte due chiavi e solo una viene tenuta segreta. La chiave pubblica è conosciuta da tutti, mentre la chiave privata non è condivisa con nessuno; la relazione è che un messaggio codificato con una delle due chiavi può essere decodificato solo con la chiave corrispondente.

Posso criptare con la chiave privata ottenendo solo integrità e autenticazione e non ripudio, senza la confidenzialità in quanto chiunque può decriptare con la mia chiave pubblica.

Se voglio implementare firme digitali per messaggi di grosse dimensioni, firmo un piccolo pezzo del messaggio, ossia il message digest, come riportato nella domanda precedente.

Challenge/response con chiave asimmetrica

La password è una funzione matematica $F(x)$ che l'utente sa calcolare: il sistema propone un valore a caso X (challenge) e l'utente deve rispondere con il valore corretto $F(X)$ (response).

I protocolli d'autenticazione challenge/response sicuri utilizzano la crittografia, cambiano il challenge di volta in volta (nonce, messaggi mandati solo una volta) e inseriscono identificativi nei messaggi criptati. Questo permette di evitare attacchi man-in-the-middle.

Gli identificativi sono necessari perché altrimenti un tipo di attacco, chiamato **reflection attack**, diventa possibile quando l'autenticazione dev'essere bidirezionale.

~~Il server chiede all'attaccante di risolvere un challenge, quindi gli passerà un nonce N , ma se intanto Alice si sta autenticando abbiamo che nel secondo step manda un nonce al server ed è il server che deve rispondere: supponiamo che l'attaccante si metta in mezzo in questa seconda fase, blocca il messaggio di Alice e manda al server un proprio messaggio, in particolare gli manda il nonce N ricevuto dal server nell'altro flusso di comunicazione! L'attaccante riceve la soluzione al problema precedente. Perciò, nella risposta al challenge vi è sempre bisogno della propria identità.~~

Il protocollo **Needham-Schroeder** fa l'autenticazione bidirezionale con la crittografia asimmetrica in **3 step**:

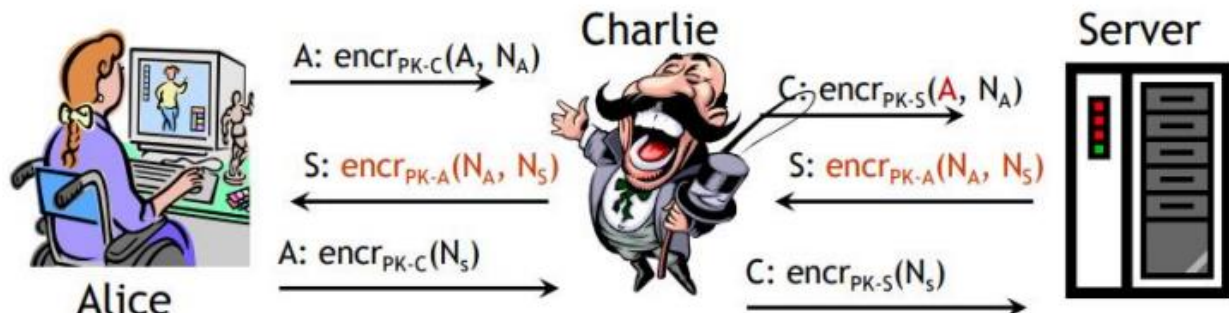
- 1) **Alice** propone un **nonce al server** criptando con la chiave pubblica del server
- 2) Il **server** scopre il nonce di Alice e nel mentre **crea un nonce: cripta queste due** informazioni con la chiave pubblica di Alice
- 3) **Alice riconosce questo nuovo nonce**, lo cripta con la chiave pubblica del server e lo rimanda, per fornire evidenza che ha scoperto il messaggio.

Quelli evidenziati sopra in giallo sono i requisiti che bisogna soddisfare: il terzo requisito qui non viene soddisfatto nel secondo e terzo messaggio, quindi è **vulnerabile**.

L'attaccante nel primo flusso riceve la richiesta di autenticazione di Alice: Alice usa la chiave pubblica dell'attaccante. L'attaccante inizia un flusso di comunicazione con il server, utilizzando lo **stesso nonce** inviato da Alice, perché è stato in grado di decriptarlo.

Il server scopre il nonce e ne manda uno pure lui; siccome bisogna dimostrare di essere Alice, il server cripta con la chiave pubblica di Alice. L'attaccante manda questa stringa criptata ad Alice.

Alice decripta e risponde all'attaccante col nonce scoperto, il quale lo manda al server!



Con gli identificativi, l'attacco non ha luogo perché Alice non si aspetta un messaggio con l'identificativo del server.

Si spieghino le differenze tra crittografia simmetrica e asimmetrica, e le implicazioni sulla relazione fra numero di chiavi e utenti per ognuno dei due casi

Nei sistemi a chiave segreta, si utilizza una sola chiave sia per criptare il dato sia per decriptarlo: due soggetti devono conoscere una chiave non nota a nessun altro.

Non si può garantire integrità e autenticità indipendentemente dalla confidenzialità.

Con N individui, per comunicare in maniera sicura servono $n(n-1)/2$ chiavi (una per coppia).

Nei sistemi a chiave pubblica, soltanto una delle due chiavi viene tenuta segreta.

Ogni soggetto possiede una coppia di chiavi, pubblica e privata.

Criptando con la chiave privata ottengo un meccanismo di sola integrità.

Per n individui c'è bisogno di n coppie di chiavi.

Cosa fa una funzione di hash? Che caratteristiche deve avere?

Un algoritmo di hashing ha l'obiettivo di creare codice di dimensione fissa (centinaia di bit) univocamente rappresentativo di un messaggio. Con message digest (una sorta di checksum crittografico) si intende il riassunto di un messaggio, idealmente ogni codice corrisponde a uno e un solo messaggio, nella pratica non è possibile ma l'algoritmo approssima questa proprietà.

Deve avere due caratteristiche:

- **non invertibile**: operazione one way, deve essere difficilissimo trovare un altro messaggio che abbia lo stesso message digest
- **collision resistant**: deve essere improbabile che due messaggi diversi producano lo stesso message digest

Perché la crittografia DES non è più considerata sicura? Quali modi di suo utilizzo potrebbero migliorare la sicurezza?

Non esiste la **prova matematica** della sicurezza di DES: si può violare con forza bruta e nel **1997** in circa **4 mesi** è stato risolto il problema in parallelo su 3500 PC.

Alternative a DES per migliorare la sicurezza sono **Triple DES** (si cifra 3 volte con 3 chiavi diverse, sicurezza pari a usare una chiave di 112 bit), oppure **AES** (permette chiavi di diversa lunghezza e si può estendere quando diventa obsoleto).

Si descriva la differenza tra DAC e MAC facendo un esempio di una politica di controllo degli accessi per ognuno dei due tipi

Discretionary Access Control: politiche per discrezione, in cui il proprietario di una risorsa può concedere l'accesso ad altri a sua discrezione.

Esempio: **Access Control List**.

Una ACL memorizza i diritti di accesso come informazione associata a ogni risorsa: data ogni risorsa memorizzo i diritti degli utenti (o gruppi) che hanno accesso.

Non vi è una rappresentazione sparsa dell'intera tabella, in quanto ogni colonna è una risorsa la cui memorizzazione avviene soltanto per i diritti definiti. Inoltre, è facile determinare gli utenti che hanno un diritto specifico su una risorsa condivisa ed è facile revocare l'accesso a una risorsa.

Mandatory Access Control: la configurazione è obbligatoriamente determinata dalle proprietà del SO secondo un access control policy con proprietà ben definite che permettono anche di proteggere rispetto ad alcuni covert channels.

Il sistema impone un modello che limita e controlla la discrezionalità degli utenti nell'assegnare i diritti di accesso alle risorse.

I modelli MAC possono essere **multi-livello** (nasce in ambito militare per garantire la confidenzialità, si classificano soggetti e oggetti in **livelli** di sicurezza e l'accesso è consentito solo se $\text{livello soggetto} \geq \text{livello oggetto}$; si usano le keyword top-secret, secret e unclassified) o **multi-lateral** (la classificazione non è più per livello ma per **gruppo di risorse** e gli utenti sono classificati come partecipanti a un certo gruppo).

Un esempio di modello MAC multi-livello è il modello di **Bell-LaPadula**, orientato alla confidenzialità. Come detto prima, ogni risorsa e utente ha un livello e valgono due proprietà:

- **proprietà di sicurezza semplice (no-read-up):** posso leggere le risorse soltanto se il mio livello di sicurezza è adeguato
- **proprietà del confinamento (no-write-down):** non posso scrivere su risorse di livello di classificazione più basso rispetto al mio livello

Il motivo è di evitare covert channels (trasmissione di dati sensibili su un canale alternativo condiviso con un soggetto non autorizzato).

Un altro modello MAC multi-livello è il modello BIBA, orientato all'integrità, in cui si invertono i concetti e si parla di livello di integrità. In questo caso valgono:

- assioma di integrità semplice (no-write-up)
- assioma di confinamento dell'integrità (no-read-down).

Si descriva il funzionamento di una Access Control List e si spieghi il concetto di “mediazione completa” nel contesto dei sistemi operativi sicuri

ACL già spiegata nella domanda precedente.

Per mediazione completa si intende controllare ogni accesso: uno dei problemi quando implemento il SO che mi devo porre è quando voglio controllare gli accessi, cioè se controllare più volte l'accesso alla stessa risorsa da parte dello stesso utente, meno efficiente ma più sicuro in caso di un attacco che supera una volta il controllo, oppure controllare solo una volta. La mediazione completa è il meccanismo più restrittivo in assoluto, ma è un principio di riferimento che nella pratica viene implementato in maniera sub-ottima a seconda del SO.

Si descriva il funzionamento di una Access Control List basata su 9 bit di protezione come quella usata nei sistemi operativi di tipo Unix/Linux

In questa implementazione si ha una maniera compatta per rappresentare una ACL, tuttavia con qualche limite sul tipo di configurazione che posso fare.

A ogni risorsa è associata una sua ACL di 9 bit: ognuno definisce se un certo diritto sia concesso per quella risorsa e i bit vanno interpretati a gruppi di 3.

I primi 3 bit parlano di diritto di R,W,X (lettura, scrittura, esecuzione) per l'**owner** della risorsa, ossia i diritti del proprietario.

I successivi 3 bit definiscono gli stessi diritti ma per il **gruppo di lavoro del proprietario**.

Gli ultimi 3 bit rappresentano i diritti di **tutti** gli utenti del SO.

Si descriva come funziona una politica di sicurezza secondo il modello di Bell-LaPadula esteso con compartimenti Si dica inoltre se tale modello definisce una politica di sicurezza di tipo MAC o DAC

Il modello di Bell-LaPadula esteso con compartimenti è un esempio di modello **MAC** multi-lateral, in cui oltre al concetto di livello si introducono i compartimenti, quindi è sia multi-livello sia multi-lateral.

Gli oggetti e i soggetti si classificano per compartimenti (ad esempio le missioni in ambito militare) e la regola che si aggiunge è la necessità di conoscere se i **compartimenti oggetto sono un sottoinsieme dei compartimenti soggetto**.

Perciò, l'accesso è consentito se $\text{livello soggetto} \geq \text{livello oggetto}$ **AND** $\text{compartimenti oggetto} \subseteq \text{compartimenti soggetto}$.

Si spieghi come funziona un firewall “packet filter”, e come può essere usato per proteggere una sottorete da attacchi IP-spoofing provenienti dall'esterno della sottorete stessa

Con un firewall packet filter (dispositivo più comune), il filtraggio avviene sui pacchetti e può essere un dispositivo HW oppure installato sul router. Di solito lavora su dati a livello 2 o 3 e monitora gli header dei pacchetti IP e TCP, prendendo decisioni a basso livello.

Le informazioni su cui si basa sono quindi: indirizzo IP sorgente e destinatario, porta TCP sorgente e destinatario e MAC address se sta operando a livello del frame.

Posso associare una politica di filtraggio definita sulla base di queste informazioni: si tratta di una sequenza di regole applicate una alla volta e sequenzialmente.

Se una regola matcha con un pacchetto, quella regola può dire se il pacchetto può essere lasciato passare o scartato (permit o deny).

Per proteggere una sottorete da **IP-spoofing provenienti dall'esterno**, il firewall può intuire una situazione di IP spoofing se vede arrivare un pacchetto dall'esterno che è indirizzato alla rete interna che però **come indirizzo sorgente ha un indirizzo della rete interna**.

Si descrivano le differenze tra firewall di tipo packet filter e firewall di tipo application gateway

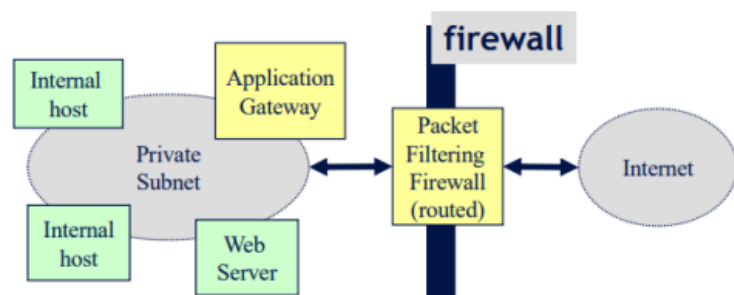
Packet filter descritto nella domanda precedente.

Per un firewall di tipo application gateway, invece, la politica di filtraggio viene definita a livello del dato applicativo, in quanto si opera sopra il livello 4. Vedo il flusso di informazione dopo che è stato ricostruito. Tipicamente è un firewall diverso per ogni applicazione, lavora al livello dei protocolli di applicazione e monitora l'application data semanticamente, mettendosi al posto dell'applicazione come proxy dell'applicazione.

Un esempio è un filtro spam per e-mail, oppure un FTP firewall configurato per accettare download ma non upload.

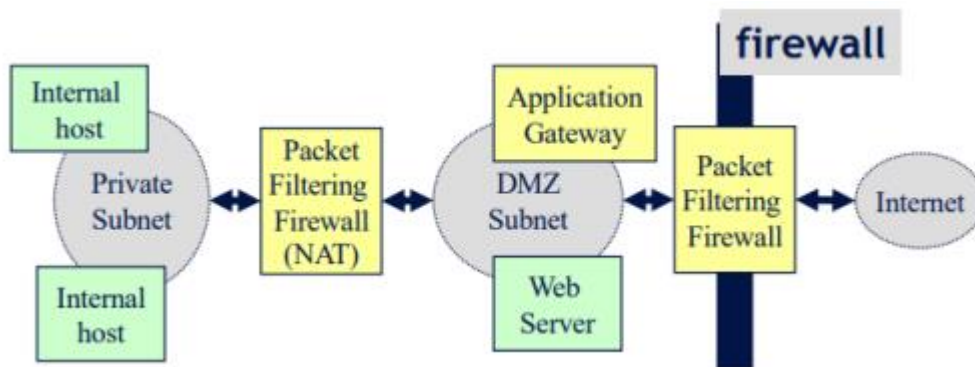
Screened subnet

Configurazione in cui i firewall application gateway e packet filter sono usati insieme per raggiungere certi obiettivi di protezione. Screened subnet è una sottorete in cui c'è un controllo su alcune applicazioni in questo modo: all'ingresso della sottorete c'è un packet filter che controlla i pacchetti e che può applicare politiche anti IP spoofing, poi per i pacchetti che passano ci possono essere alcuni servizi a loro volta protetti con un firewall di tipo application gateway.



DMZ

La parte iniziale è simile a una screened subnet: packet filter all'ingresso, poi, se la nostra azienda fornisce dei servizi all'esterno ma allo stesso tempo ha una rete interna per gli impiegati, si partiziona la sua sottorete in due sotto-sottoreti: una immediatamente accessibile dopo un firewall application gateway, dopodiché la parte delle macchine interne può essere separata da un altro firewall che lavora di nuovo a livello IP, con una politica di filtraggio più stretta (es. tutti gli indirizzi interni della sottorete non sono noti all'esterno, meccanismo di Network Address Translation).



Si descriva la struttura di un pacchetto IP trasmesso usando il protocollo SSL, spiegando che tipo di protezione di ottiene

Sulle slide parla del successore TLS, quindi qui parlo di TLS.

Il protocollo Transport Layer Security è posto al di sopra di TCP, quindi prima di aggiungere informazioni a livello TCP un'applicazione che trasmette dei dati aggiungerà informazioni che servono per garantire la **confidenzialità** e l'**integrità** dell'application data: il dato applicativo viene criptato.

Un pacchetto costruito usando TLS avrà questa struttura: **dati applicativi criptati**, header TCP e IP e dati aggiuntivi che servono a TLS per gestire il meccanismo di decryption (es. versione, algoritmo utilizzato e lunghezza del dato).



5 bytes:
• type (1 byte),
• version (2 byte),
• length (2 byte)

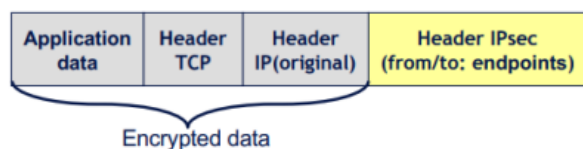
Il protocollo TLS è composto da due parti, cioè Handshake Protocol e Application Data Protocol.

- **Handshake: Hello Phase**
 - o La prima comunicazione avvisa l'altra con un messaggio di client_hello e il server risponde con server_hello
 - o Si scambiano ad esempio la versione di TLS e la cypher-suite
- **Handshake: Server Authentication Data**
 - o Il server manderà un suo certificato al client: gestendo con RSA la comunicazione, il certificato permette al client di verificare l'identità del server e anche di ottenere la sua chiave pubblica
 - o Opzionale: il server può richiedere un certificato al client
- **Handshake: Client Key Exchange**
 - o Il client, conoscendo la chiave pubblica del server e fidandosi, manda un numero random criptato con la chiave pubblica del server, che sarà usato come **chiave di sessione**
- **Handshake: Finish**
 - o A questo punto vi è uno scambio di due ulteriori messaggi, uno in cui si ha la conferma che è stata impostata la cypher-suite e uno chiamato **encrypted handshake** perché viene mandato l'hash di tutti i messaggi scambiati fino a quel momento, criptato con la chiave condivisa
- **Application Data sub-protocol**
 - o A questo punto l'handshake è concluso e possono mandarsi i dati applicativi, in quanto entrambi conoscono la chiave di sessione

Chiunque intercetta i messaggi vede soltanto dei dati criptati, tuttavia la **sicurezza** viene aggiunta **solo sopra il livello di trasporto** e quindi gli header TCP e IP sono visibili.

Si descriva la struttura di un pacchetto IP trasmesso usando il protocollo IPsec in modalità tunneling, specificando il tipo di protezione che si ottiene

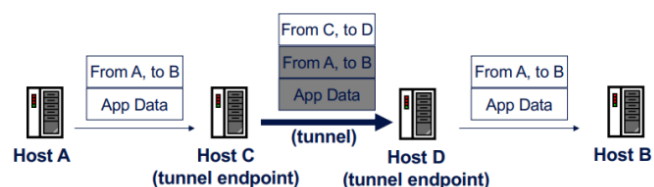
La crittografia viene applicata a livello IP: c'è uno strato IPsec, sotto IP, che si occupa di criptare l'informazione dei dati applicativi, della parte TCP e anche la parte IP.



Si aggiunge un altro indirizzo IP per permettere la trasmissione del pacchetto.

La configurazione avviene tra punti intermedi che si chiamano **tunnel endpoint**. Nel momento in cui il pacchetto esce dalla mia sottorete, viene criptato e questo router dichiara di inoltrarlo a un altro router. Chi intercetta questo pacchetto nella strada intermedia non sa quali sono sorgente e destinazione originali.

L'ultimo tunnel endpoint decriptando il contenuto sa l'host di destinazione e lo inoltra.



Si descriva il funzionamento di un attacco XSS

Si tratta di un attacco di tipo **code injection**, si verifica quando un attaccante riesce a eseguire codice malevolo (es. Javascript) all'interno del browser di un utente, tramite una vulnerabilità di un sito web. Ciò permette il furto di cookie, keylogging, ecc.

Si hanno tre tipologie di attacchi XSS:

- **Persistent XSS**
 - L'attaccante usa una form del sito web per inserire codice malevolo nel **database** del sito
 - La vittima richiede una pagina
 - Il sito web include nella pagina il codice malevolo e invia la risposta alla vittima
 - Il browser della vittima esegue lo script malevolo
- **Reflected XSS**
 - L'attaccante crea un URL contenente una stringa malevola e lo invia alla vittima ad esempio via e-mail
 - La vittima clicca sull'URL
 - Il sito web invia una pagina di risposta che include la stringa malevola
 - Il browser della vittima esegue il codice malevolo
- **DOM-based XSS**
 - Come prima, l'attaccante crea un URL contenente una stringa malevola e lo invia alla vittima
 - La vittima clicca sull'URL
 - Il sito web invia una pagina dinamica che non include il codice malevolo ma codice Javascript vulnerabile
 - Il codice Javascript incluso nella pagina
 - Estrae il codice malevolo dall'URL
 - Lo aggiunge dinamicamente alla pagina
 - Il browser dell'utente interpreta lo script contenente il codice malevolo

Si descriva il funzionamento di un attacco SQL injection

Si tratta di uno schema d'attacco di tipo **command injection**: ha l'obiettivo di modificare l'esecuzione di un comando SQL parametrico all'interno dell'applicazione.

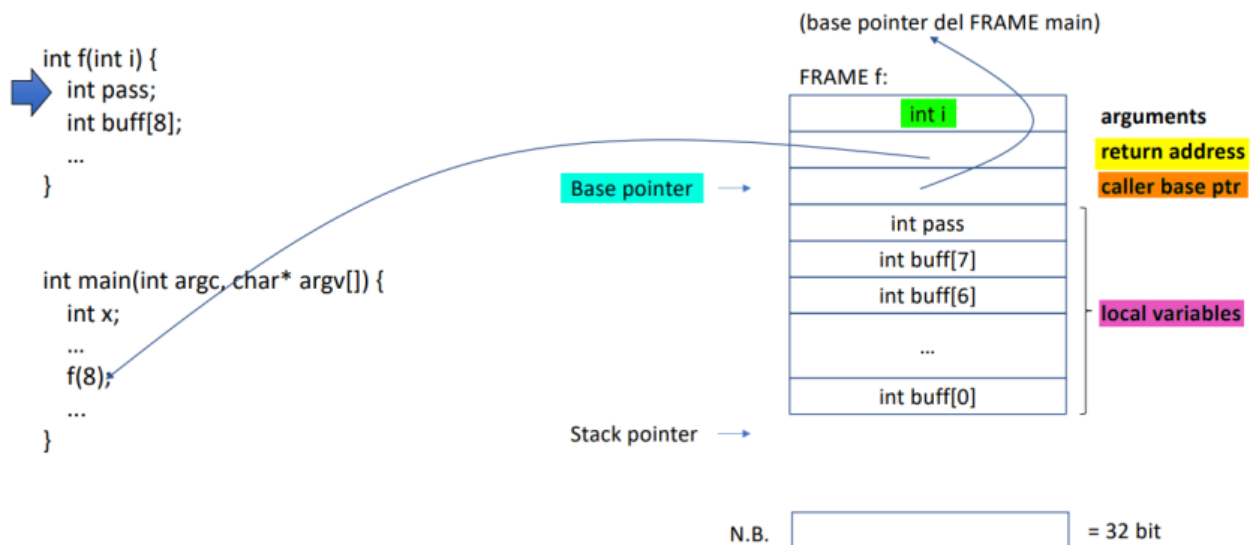
Consiste nell'inserimento nel sistema di una porzione di query SQL attraverso l'input utente. Se eseguito con successo, ad esempio può accedere a dati sensibili, eseguire operazioni di amministrazione del DB, leggere il contenuto di un file presente nel filesystem del DBMS.

Si può manifestare in tutti i programmi che si interfacciano con DBMS, sono disponibili librerie di encoding per ovviare al problema.

Si descriva il funzionamento di un attacco di tipo buffer overflow

Con buffer overflow si intende la scrittura fuori dai limiti (out-of-bound) di un buffer, possibile in un linguaggio come C. Ciò può essere interessante per un attaccante quando il buffer si trova tra le variabili locali di un programma, in quanto nell'activation frame abbiamo il **return address**. È necessario modificare anche il base pointer (per evitare

segmentation fault), che ci dice dove finisce la parte di stack preparata dal main e dove inizia la parte di stack preparata dalla funzione f.



Con `buff[8] = 97` si ha una scrittura out-of-bound e si sovrascrive la variabile `pass`.

Importante notare come `buff[10]` corrisponde alla regione che contiene il return address. Se la sovrascriviamo, possiamo far sì che l'invocazione di `f` termini saltando ad una funzione da noi scelta.

L'attaccante riempie un buffer con codice malevolo ("shellcode", una chiamata di sistema `execve`) e sovrascrive il return address del frame corrente, per passare ad eseguire il codice malevolo.

In questo modo l'attaccante ottiene una shell con privilegi di superuser.

Lo shellcode viene circondato da NOP per permettere salti imprecisi.

Come BP si può usare il BP del chiamante, individuato tramite un debugger.

Buffer overflow può anche riguardare buffer allocati sullo heap, ma ovviamente non si può riscrivere il return address. Però, si possono riscrivere puntatori a funzione.

Come contromisure abbiamo:

- **Canary values:** a ogni invocazione di funzione viene inserito un valore di controllo sullo stack, dopo il BP. Si tratta di un valore inserito dal compilatore e salvato in un'area lontana dallo stack. Viene controllato all'uscita dalla funzione e dev'essere identico al valore inserito, in caso contrario il programma termina con un messaggio di errore. Il programma non deve permettere letture out-of-bound
- **ASLR:** Address Space Layout Randomization, alcuni indirizzi del virtual address space vengono modificati da un'esecuzione ad un'altra, rendendo difficile la scrittura di exploit
- **Executable Space Protection:** il SO distingue aree di memoria contenente codice da aree di memoria contenente dati, nei processori con MMU si usa un bit di protezione nella tabella delle pagine per indicare se la pagina è eseguibile.